K. J. SOMAIYA COLLEGE OF ENGINEERING
DEPARTMENT OF INFORMATION TECHNOLOGY / COMPUTER ENGINEERING

Automated PostgreSQL Backup and Point-in-Time Recovery using pgBackRest

Automating Database Backups, WAL Archiving, and Disaster Recovery without Cloud Infrastructure

Course: DevOps Theory (IA Case Study)	**Date:** 2026-10-18
Team Members: • Aaryan Sharma (16010123012) • Anamay Narkar (16010123040) • Aditya Singh (16010123025) • Vraj Soni (160101220)	**Faculty / Evaluator:** Department of Information Technology / DevOps Evaluator **Stack:** PostgreSQL 17.11 • pgBackRest 2.59.3
Application: StagePass Event Operations	**Evaluation Host:** Docker Compose (Zero Cloud Dependency)
Flagship Scenario: KJSCE Garba Eve (12 Oct)	**Reproducibility:** Deterministic 100-Row Seed + 20 HTTP Workload

1. Executive Summary & Abstract

This project demonstrates how pgBackRest automates PostgreSQL database backups, WAL archiving, and disaster recovery. It explores restoring a database to a specific point in time after accidental data loss, evaluating recovery time and reliability without requiring cloud infrastructure.

In enterprise database systems, operator error—specifically unconstrained DELETE or UPDATE statements—represents one of the most severe classes of production downtime. This case study investigates disaster recovery mechanisms for PostgreSQL 17.11 through an empirical, end-to-end event registration platform called **StagePass**. We contrast a traditional stopped-cluster filesystem snapshot (rsync) against **pgBackRest 2.59.3** with continuous Write-Ahead Log (WAL) archiving and Point-in-Time Recovery (PITR).

Under identical workloads of 100 seeded rows and 20 real HTTP bookings, an accidental DELETE removes all 40 flagship bookings. The snapshot-only baseline recovers the cluster to snapshot time (100 bookings), but completely loses all 20 post-snapshot transactions because those transactions occurred after the snapshot and the baseline deliberately has no WAL replay mechanism. In contrast, in both recorded PITR trials, all 20 post-backup registrations were recovered by restoring to the verified pre-incident named restore point (stagepass_before_incident_pgbackrest_run2). This experiment achieved zero loss of the 20 tested registrations; production RPO depends on WAL archival and retention. Observed recovery time (RTO) was 17.6 s in Run 1 and 20.6 s in Run 2 (differences of 2.6 s and 10.0 s compared to snapshot-only). Furthermore, differential and incremental backup strategies demonstrated an 85% reduction in individual backup storage footprint and over 70% reduction in execution time compared to full backups.

[KEY CONTROLLED FINDING]

In both recorded PITR trials, all 20 post-backup registrations were recovered by restoring to the verified pre-incident named restore point. This experiment achieved zero loss of the 20 tested registrations; production RPO depends on WAL archival and retention.

2. Introduction & Industrial Relevance

PostgreSQL serves as the transactional backbone across microservice architectures, enterprise SaaS, and mission-critical web platforms. While Site Reliability Engineering (SRE) paradigms emphasize High Availability (HA) through replication, replication alone does not protect against data corruption: a bad DELETE command immediately propagates to all read replicas within milliseconds. True disaster recovery requires historical immutability via write-ahead log replay. This project explores local disaster recovery automation, demonstrating that robust point-in-time recovery, continuous WAL archiving, and backup retention can be achieved reliably without requiring cloud infrastructure.

3. Problem Statement

During peak traffic for the KJSCE Garba Eve event on 12 October, an operational incident occurs: an engineer inadvertently issues an unbounded query: DELETE FROM bookings WHERE event_id = 1;. The Garba booking count drops to 0, and the operations dashboard visibly reports service degradation. The engineering objectives are: (1) automate restoration of database state to a specific point in time, (2) evaluate recovery time (RTO) and reliability (RPO) without relying on external cloud infrastructure, and (3) quantify the time and storage trade-offs across backup modes.

4. Research Question & Methodological Objectives

Project Focus: *This project demonstrates how pgBackRest automates PostgreSQL database backups, WAL archiving, and disaster recovery. It explores restoring a database to a specific point in time after accidental data loss, evaluating recovery time and reliability without requiring cloud infrastructure.*

- **Objective 1 (Automated Lab):** Build a complete, containerized environment with FastAPI, Jinja2, PostgreSQL 17, and isolated storage volumes.
- **Objective 2 (Zero-Cloud Disaster Recovery):** Automate PostgreSQL backups, WAL archiving, and Point-in-Time Recovery to a target point without cloud infrastructure.
- **Objective 3 (Recovery Time & Reliability):** Subject both recovery architectures to identical traffic workloads, seed states, and incident triggers; measure RTO and RPO.
- **Objective 4 (Empirical Trade-off Matrix):** Measure execution duration and compressed repository storage across Full, Differential, Incremental, and Logical (pg_dump) strategies.

5. System Architecture & Volume Isolation

The experimental lab runs via Docker Compose, segregating concerns across isolated containers and volumes:

1. **App Container (StagePass):** FastAPI with Jinja2 templates, serving public registration and operations monitoring endpoints. Uses short-lived database connections to survive database restarts.
2. **DB Container:** PostgreSQL 17.11 (Host Port 5433, Container Port 5432) configured with wal_level=replica and archive_mode=on. Runs pgBackRest 2.59.3 directly for low-latency WAL pushing.
3. **Backup-Tools Container:** Operational sidecar with volume mounts for pgdata, repo, and snapshots. Executes controlled rsync snapshots and restore procedures.
4. **Storage Isolation:** The pgBackRest repository (/var/lib/pgbackrest) resides on a dedicated Docker volume separate from PGDATA, guaranteeing that a destructive restore never overwrites backup archives.

6. Theoretical Principles: Physical vs Logical Backups & WAL Replay

Logical Backups (pg_dump): Logical backups dump database DDL and DML into SQL or custom binary archives. While highly portable and selective, they cannot replay write-ahead logs. In our benchmark, pg_dump completed in 0.3 s (0.5 MB), but can only restore the state at dump completion.

Physical Backups + WAL (pgBackRest): Copies raw 8 KB data pages alongside a continuous stream of 16 MB WAL segments. When PostgreSQL boots in recovery mode, it replays WAL records forward from the backup checkpoint LSN up to the requested target. This enables Point-in-Time Recovery to a verified named restore point.

7. Controlled Experimental Results (A/B Comparison)

Both scenarios were executed through automated, guarded test harnesses and replicated across two independent trials (Run 1 and Run 2):

Run Identifier	Baseline	Pre-Incident	Recovered	New Regs	Backup Time	Restore (RTO)
rsync-run1	100	120	100	0	1.1 s	15.0 s
pgbackrest-run1	100	120	120	20	13.3 s	17.6 s
rsync-run2	100	120	100	0	0.5 s	10.6 s
pgbackrest-run2	100	120	120	20	7.2 s	20.6 s

8. Backup Strategy Trade-off Benchmarks (20,000 Rows - Single Trial)

Benchmarked on a synthetic 20,000-row dataset to evaluate time and storage compression across strategies (distinguishing individual backup size from cumulative repository total):

Trial	Backup Mode	Rows	Duration (s)	Backup Size	Cumulative Repo	Notes
1	FULL	20000	10.6 s	4.50 MiB	11.58 MiB	Base physical copy (100% data)
1	DIFFERENTIAL	20000	2.5 s	697.7 KiB	13.67 MiB	5% rows modified (id%20==0)
1	INCREMENTAL	20000	3.0 s	635.8 KiB	15.53 MiB	5% rows modified (id%20==7)

Note: The benchmark table occupied approximately 3.9 MiB of PostgreSQL relation storage (pg_total_relation_size('bench_payload') ≈ 3,896 KiB); pgBackRest reported a 33.5 MB database size for the cluster used in the backup measurement. Individual backup size measures the new data delta (Full: 4.50 MiB, Diff: 697.7 KiB, Incr: 635.8 KiB). Cumulative repository size reflects total disk footprint measured via du -sb /var/lib/pgbackrest immediately post-backup: Full = 11.58 MiB (12.1 MB decimal; 12,137,779 bytes), Diff = 13.67 MiB (14.3 MB decimal; 14,338,561 bytes), Incr = 15.53 MiB (16.3 MB decimal; 16,284,776 bytes).

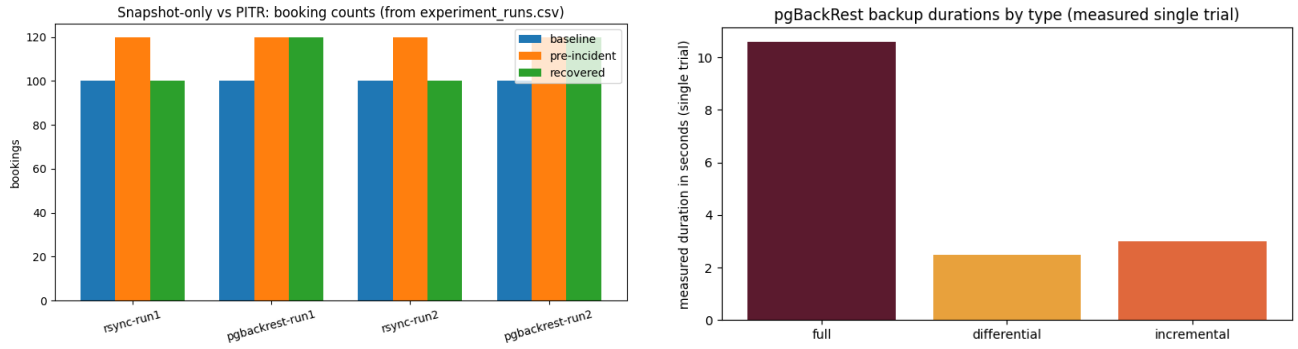


Figure 1: Empirical booking recovery counts (Left) and backup duration benchmarks (Right).

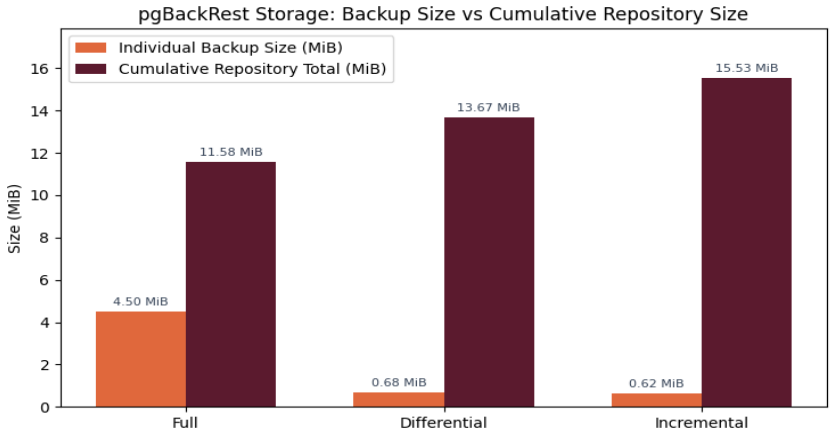


Figure 2: Storage comparison — Individual backup size vs Cumulative repository total.

9. Synthesis, Discussion & Limitations

RTO vs RPO Trade-offs: While rsync snapshot recovery restored the cluster in 10.6 s & 15.0 s, it lost the 20 post-snapshot registrations because those transactions occurred after the snapshot and the baseline deliberately has no WAL replay mechanism. PITR required longer recovery times, with observed differences of 2.6 seconds (Run 1: 17.6 s vs 15.0 s) and 10.0 seconds (Run 2: 20.6 s vs 10.6 s). Two runs illustrate the trade-off between snapshot-only restoration and full WAL replay to a verified target. In both recorded PITR trials, all 20 post-backup registrations were recovered by restoring to the verified pre-incident named restore point. This experiment achieved zero loss of the 20 tested registrations; production RPO depends on WAL archival lag and retention availability.

Threats to Validity & Limitations:

1. *Local Storage Bias:* The pgBackRest repository was mounted locally on Docker volumes rather than remote cloud object storage. Network latency in production will increase transfer times.
2. *Single Trial Benchmark:* Backup mode benchmark figures represent a single controlled trial rather than a multi-trial statistical median.
3. *Archive Lag & Retention:* The configured archive timeout is 60 seconds. Actual recoverability depends on WAL archival success, archive lag, shipping latency, and retention. The experiment verified that the target segment was available before restoration.

10. Primary Documentation & Academic References

[1] pgBackRest Development Team, 'pgBackRest User Guide & Architecture Documentation', v2.59.3, https://pgbackrest.org/user-guide.html (Accessed Oct 2026).

[2] PostgreSQL Global Development Group, 'PostgreSQL 17.11 Documentation: Continuous Archiving and Point-in-Time Recovery (PITR)', Chapter 25, https://www.postgresql.org/docs/17/continuous-archiving.html.

[3] PostgreSQL Global Development Group, 'Backup and Restore Strategies: Logical vs Physical Backups', https://www.postgresql.org/docs/17/backup.html.

[4] Beyer, B., Jones, C., Petoff, J., & Murphy, N. R., 'Site Reliability Engineering: How Google Runs Production Systems', O'Reilly Media.

[5] Kleppmann, M., 'Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems', O'Reilly Media.